

Xây dựng Data Lakehouse phục vụ phân tích thị trường chứng khoán HOSE: thiết kế pipeline, streaming và alerting từ góc nhìn Data Engineer

[HỌ VÀ TÊN SINH VIÊN], [HỌ VÀ TÊN MENTOR]

[ĐƠN VỊ MENTOR HƯỚNG DẪN]

Email mentor: [EMAIL MENTOR HƯỚNG DẪN]

Abstract—Để tài xây dựng một Data Lakehouse cho dữ liệu chứng khoán niêm yết trên HOSE, bao trùm toàn bộ vòng đời dữ liệu: thu thập đa nguồn, xử lý batch theo ba lớp Bronze-Silver-Gold, một luồng streaming thời gian thực qua Kafka Engine của ClickHouse, và một Alert Engine theo mô hình event-driven tách cấu hình khỏi lịch sử sự kiện. Hệ thống lưu Parquet ba lớp trên MinIO, dùng ClickHouse làm lớp serving phân tích, Airflow điều phối DAG batch 16 task mỗi ngày, và vận hành thêm hai tiến trình nền độc lập: alert-engine quét điều kiện cảnh báo mỗi 60 giây trên toàn bộ vũ trụ 403 mã HOSE, và realtime-vwap-consumer tổng hợp trade tick nhận qua Kafka thành VWAP phiên lũy kế mỗi 15 giây. Tại thời điểm đánh giá 28/06/2026, Gold lưu 403 mã HOSE, 12.963 bản ghi giá từ 12/12/2024 đến 26/06/2026, 66 bản ghi chỉ số VNINDEX/VN30 (VN30 lấy đúng 30 mã thành viên chính thức thay vì quy tắc mô phỏng theo vốn hóa), 640 bản ghi cảm xúc tin tức và hàng nghìn sự kiện cảnh báo đã ghi log qua cơ chế chống spam theo cooldown. Bộ kiểm thử tự động gồm 164 test, toàn bộ pass; công cụ lint tĩnh không còn lỗi. Một dashboard Grafana giám sát vận hành (pipeline freshness, độ trễ Kafka, trạng thái cảnh báo, lịch sử Airflow DAG run) được provision hoàn toàn tự động bằng cấu hình, bổ sung cho dashboard Superset phục vụ phân tích nghiệp vụ. Trong quá trình rà soát hệ thống đang chạy liên tục, sinh viên phát hiện và sửa một lỗi vận hành thật: cơ chế cooldown của Alert Engine vô hiệu khi chưa cấu hình kênh gửi, khiến bảng sự kiện tăng không kiểm soát theo từng vòng quét; lỗi được sửa và xác nhận lại bằng theo dõi log trực tiếp. Đóng góp chính của sinh viên tập trung vào các quyết định ở vai trò data engineer: chiến lược phân vùng theo grain truy vấn riêng cho từng tập dữ liệu, thiết kế idempotent cho loader Gold và cho streaming consumer (tái tính theo partition thay vì append không kiểm soát), tách lớp lưu trữ lâu dài (MinIO) khỏi lớp serving tốc độ cao (ClickHouse), một pipeline cảnh báo tách bằng cấu hình giao dịch cao (PostgreSQL) khỏi bảng sự kiện chỉ thêm

theo thời gian (ClickHouse), và một lớp giám sát vận hành tách biệt khỏi lớp phân tích nghiệp vụ. Báo cáo cũng trình bày rõ giới hạn còn lại: kênh gửi cảnh báo qua email, mô hình sentiment học máy cho tin tức và một số hạn chế vận hành cụ thể.

Index Terms—Data Lakehouse, HOSE, Airflow, MinIO, ClickHouse, Kafka, Parquet, ETL idempotent, Event-driven alerting, VWAP, Grafana, Observability.

I. Giới thiệu chung

A. Bối cảnh và nhu cầu

Dữ liệu chứng khoán đến từ nhiều nguồn và có đặc điểm không đồng nhất: giá giao dịch dạng chuỗi thời gian, hồ sơ doanh nghiệp thay đổi chậm, chỉ số thị trường tổng hợp, tin tức dạng văn bản và trade tick realtime có tần suất cao. Nếu chỉ tải dữ liệu về các tệp cục bộ, người dùng khó xác định phiên bản mới nhất, khó phát hiện bản ghi lỗi, không có cơ chế cảnh báo khi điều kiện bất thường xảy ra, và phải lặp lại nhiều thao tác trước mỗi lần phân tích. Bài toán thực tiễn đặt ra là xây dựng một nền tảng dữ liệu thống nhất, tự động cập nhật theo lịch, lưu được dữ liệu gốc để đối soát, cung cấp dữ liệu đã xử lý cho dashboard với độ trễ phù hợp, và có khả năng phản ứng theo thời gian gần thực khi cần.

Phạm vi hiện tại được giới hạn ở cổ phiếu niêm yết trên HOSE. Việc giới hạn này giúp tập trung hoàn thiện chất lượng dữ liệu và tính nhất quán xuyên suốt pipeline trước khi mở rộng sang sàn khác. Hai chỉ số đại diện được theo dõi là VNINDEX và VN30, trong đó VN30 dùng đúng danh sách 30 mã thành viên chính thức.

B. Mục tiêu và đóng góp của sinh viên

Mục tiêu của đề tài là: (i) thu thập tự động dữ liệu đa nguồn; (ii) tổ chức dữ liệu theo kiến trúc Bronze-Silver-Gold với chiến lược phân vùng phù hợp từng tập dữ liệu; (iii) kiểm soát schema và chất lượng trước khi nạp Gold; (iv) bổ sung một luồng streaming thật cho dữ liệu realtime thay cho việc nạp trực tiếp; (v) xây dựng một pipeline cảnh báo theo điều kiện kỹ thuật có khả năng chống spam; và (vi) cung cấp API, dashboard và báo cáo phân tích theo dõi thị trường, cổ phiếu và VWAP.

Sinh viên trực tiếp khảo sát nguồn, thiết kế kiến trúc, hiện thực các bộ thu thập và biến đổi bằng Python/Polars, xây dựng DDL ClickHouse và mô hình dbt, cấu hình Airflow, đóng gói hạ tầng bằng Docker Compose, phát triển FastAPI, giao diện React và dashboard Superset. Các điểm cải tiến nổi bật ở vai trò data engineer gồm: (1) tách kho Parquet lâu dài khỏi lớp serving — cả ba lớp được lưu trên MinIO, ClickHouse chỉ đảm nhiệm truy vấn tốc độ cao; (2) chiến lược phân vùng được lựa chọn theo grain và cách truy vấn của từng tập dữ liệu thay vì áp dụng một quy tắc chung; (3) một luồng Kafka Engine thật cho realtime VWAP, thay thế hoàn toàn cách nạp trực tiếp dữ liệu demo vào bảng serving; và (4) một Alert Engine tách bảng cấu hình (PostgreSQL, có tính giao dịch cao) khỏi bảng lịch sử sự kiện (ClickHouse, chỉ thêm theo thời gian), kèm cơ chế chống spam có thể kiểm chứng.

II. Nội dung và phương pháp

A. Nguồn và phạm vi dữ liệu

Hệ thống sử dụng thư viện vnstock để truy cập dữ liệu thị trường Việt Nam [1]. Danh sách mã HOSE lấy qua Listing với nguồn KBS; lịch sử OHLCV từng mã lấy qua Quote với nguồn VCI. Hồ sơ doanh nghiệp được lấy qua Company/KBS. Dữ liệu VNINDEX và VN30 cũng lấy qua Quote/VCI; riêng thành phần VN30 dùng trực tiếp Listing().symbols_by_group("VN30") của vnstock để có đúng 30 mã thành viên chính thức, có cơ chế dự phòng quay về quy tắc top-30 theo vốn hóa khi nguồn này không phản hồi. Mô-đun realtime xác thực DNSE WebSocket bằng API key/secret và HMAC-SHA256, sau đó chuẩn hóa trade tick và có thể publish trực tiếp lên Kafka topic dnse-trades-raw khi chạy với cờ --produce-to-kafka.

B. Xử lý chi tiết nguồn tin tức

Crawler được viết riêng cho cấu trúc HTML của từng báo thay vì dùng một selector chung. Mọi request sử dụng một Session, header mô phỏng trình duyệt tiếng Việt và timeout 25 giây. Một lỗi

TABLE I
Nguồn, nhịp cập nhật và đầu ra chính

Dữ liệu	Nguồn	Nhịp xử lý	Gold
OHLCV HOSE	VCI	Sau phiên, thứ Hai-Sáu	fact_daily_price dim_stock
Doanh nghiệp Chỉ số VN30	KBS + VCI/Listing	Ngày/tuần Sau phiên	fact_market_index
Tin tức	3 báo điện tử	Cấu hình 30-60 phút; DAG ngày	fact_news_sentiment_daily
Trade tick	DNSE qua Kafka	Trong giờ thị trường hoặc demo	fact_realtime_vwap
Cảnh báo	user_alerts (Postgres)	Liên tục, 60 giây/lần	fact_alert_event

HTTP hoặc lỗi parse chỉ làm bỏ qua bài tương ứng, không dừng toàn bộ nguồn. Trong mỗi lần chạy, URL được khử trùng bằng seen_urls; hệ thống chỉ nhận bài tối đa hai ngày tuổi và giới hạn 20 bài cho mỗi nguồn.

VnExpress: crawler duyệt bảy chuyên mục Kinh doanh gồm Vĩ mô, Doanh nghiệp, Ebank, Quốc tế, Hàng hóa, Kinh tế vùng và Chứng khoán. Từ trang chuyên mục, link được lấy tại thẻ h2/h3.title-news. Ở trang bài viết, tiêu đề lấy từ h1.title-detail, mô tả từ p.description, còn nội dung là các đoạn p.Normal trong article.fck_detail. Khối tin liên quan được loại trước khi ghép nội dung. Tags ưu tiên vùng tags của trang, nếu không có thì đọc meta keywords; ngày đăng ưu tiên meta pubdate, sau đó mới dùng span.date.

Vietstock: thay vì quét trực tiếp trang danh mục, hệ thống gửi POST tới StartPage/ChannelContentPage theo chín mã kênh: Chứng khoán, Doanh nghiệp, Bất động sản, Phân tích, Tài chính cá nhân, Tài chính, Kinh tế, Đông Dương và Thế giới. Mỗi request lấy trang đầu với pageSize=15. Link chỉ được nhận nếu kết thúc bằng .htm, có ID số và không thuộc fili.vn. Do Vietstock có nhiều template, bộ trích xuất dùng chuỗi selector dự phòng cho title, sapo và content. Script loại script, style, box liên quan; sau đó ghép các thẻ p, đọc tags hoặc meta keywords, và lấy thời gian từ article:published_time/vùng date.

CafeF: crawler duyệt năm chuyên mục Chứng khoán, Doanh nghiệp, Ngân hàng, Vĩ mô và Bất động sản. Link hợp lệ phải là bài .chn có ID số; video, tác giả, tag, emagazine, longform, megastory, infographic và photo bị loại ngay ở bước discovery. Các template đặc biệt như video de-

tail, sponsored detail và emagazine cũng bị bỏ qua để tránh tạo bản ghi thiếu nội dung. Với bài chuẩn, title lấy từ h1.title, sapo từ h2.sapo, content từ cafef_detail_tag/detail-content; script, style, media nhúng, tin liên quan và footer link được loại trước khi ghép paragraph. Tags được tìm qua nhiều wrapper dự phòng và làm sạch các nhãn "Tags", "Từ khóa".

Ba nguồn sau đó được đưa về cùng schema gồm URL, tiêu đề, ngày đăng, mô tả, nội dung, tags, nguồn, chuyên mục và thời điểm crawl. Text được co khoảng trắng, ngày được chuẩn hóa về YYYY-MM-DD; bài thiếu title hoặc content bị loại. Trong Bronze, dữ liệu cùng ngày được append rồi deduplicate theo URL và ghi tại news/source=multi/year/month/day/data.parquet; giá trị cột source dùng để phân biệt từng báo.

Sang Silver, pipeline đọc toàn bộ Bronze news, chuẩn hóa tên/kiểu cột, ép published_at về Date, loại URL/title rỗng, nội dung ngắn hơn 50 ký tự và trùng URL; kết quả được sắp xếp theo nguồn, ngày, tiêu đề rồi ghi theo news/year/month. Quality gate tiếp tục kiểm tra URL duy nhất, title, content, source và ngày đăng không null.

Để gắn tin với cổ phiếu HOSE, hệ thống tạo từ điển alias từ ticker, tên doanh nghiệp tiếng Việt/Anh và tên đã bỏ các tiền tố như "CTCP", "Ngân hàng TMCP", "Tổng Công ty". Alias quá chung bị loại; ticker ngắn được match bằng regex có biên ký tự để giảm nhầm lẫn. Match ticker có điểm 2, match tên doanh nghiệp có điểm 1; khóa article_id--ticker được khử trùng. Cuối cùng, tiêu đề được chấm sentiment bằng từ điển tích cực/tiêu cực, rồi tổng hợp theo ticker-ngày thành số bài, số nguồn, số tin tích cực/tiêu cực/trung tính, điểm trung bình và headline đại diện trong fact_news_sentiment_daily. Đây là baseline rule-based, không phải mục tiêu chính của đề tài; phần mô hình NLP học máy nằm ngoài phạm vi báo cáo này vì trọng tâm là hạ tầng dữ liệu.

C. Kiến trúc và luồng xử lý

Kiến trúc triển khai được minh họa ở Hình 1. Docker Compose cung cấp 12 service: MinIO, ClickHouse, PostgreSQL, Kafka/ZooKeeper, Airflow (webserver + scheduler), Superset, Grafana, một service NLP serving độc lập, và hai tiến trình nền tự viết — alert-engine và realtime-vwap-consumer — chạy liên tục bằng restart: unless-stopped, độc lập với lịch của Airflow. PostgreSQL lưu cả metadata của Airflow và bảng cấu hình cảnh báo user_alerts.

Bronze lưu dữ liệu gần nguyên bản để tái xử lý và đối soát. OHLCV được phân

vùng ticker/year/month/day; chỉ số theo index_code/year/month/day; tin tức hiện gom ba báo theo source=multi/year/month/day. Cách chia này phù hợp với ingest theo mã hoặc theo ngày chạy crawler.

Silver chuẩn hóa tên cột, kiểu dữ liệu, mã chứng khoán, ngày giao dịch và loại bỏ trùng lặp. OHLCV được gom theo ticker/year/month, giúp đọc lịch sử một mã mà không mở toàn bộ thị trường. Profile, index và news được phân vùng theo tháng. Tập mã đầu vào của Silver được lấy từ listing đã lọc HOSE, tránh tiếp tục xử lý các thư mục cũ thuộc sàn ngoài phạm vi.

Gold sử dụng mô hình chiều gồm dim_date, dim_stock, dim_sector, dim_index và năm fact, cộng thêm fact_alert_event cho luồng cảnh báo. Dimension xuất Parquet theo snapshot_date; các fact ngày được chia year/month; VWAP chia theo trading_date; fact_alert_event chia theo toYYYYMM(triggered_at). ClickHouse dùng MergeTree và khóa sắp xếp theo mã/thời gian để phục vụ lọc, tổng hợp và vẽ chuỗi thời gian [3]. Sau khi nạp, Gold được xuất ngược về MinIO để bảo đảm lớp Gold không chỉ tồn tại trong cơ sở dữ liệu serving.

D. Streaming thời gian thực qua Kafka Engine

Phiên bản đầu của luồng realtime nạp trực tiếp dữ liệu demo vào fact_realtime_vwap bằng Python, không qua Kafka. Để có một luồng streaming thật, hệ thống được thiết kế lại theo hai giai đoạn tách biệt nhằm tránh lỗi tính sai VWAP phiên lũy kế khi cố gắng tổng hợp trực tiếp trong Materialized View của ClickHouse.

Giai đoạn một là một passthrough đơn giản và an toàn: bảng kafka_realtime_trade_ticks dùng ENGINE = Kafka đọc topic dnse-trades-raw theo JSONEachRow; một Materialized View chỉ sao chép nguyên trạng (ticker, trade_ts, price, volume) vào bảng MergeTree realtime_trade_ticks_raw, không thực hiện phép tổng hợp nào ở tầng SQL. Giai đoạn hai là một tiến trình Python (realtime-vwap-consumer) chạy mỗi 15 giây: đọc lại toàn bộ tick của ngày hiện tại từ bảng raw, tái sử dụng đúng hàm build_fact_realtime_vwap (đã được kiểm thử ở mô-đun demo) để tính VWAP một phút và VWAP phiên lũy kế bằng cum_sum theo cặp (ticker, trading_date), sau đó DROP PARTITION của ngày hiện tại trước khi ghi lại. Cách làm này cho kết quả idempotent: chạy lại nhiều lần trên cùng dữ liệu không tạo bản ghi trùng, khác với việc chỉ INSERT thêm.

Thiết kế này đã được kiểm chứng end-to-end: publish 60 trade tick demo của ba mã VCB/FPT/HPG lên topic Kafka; Kafka Engine và

Nguồn dữ liệu (vnstock/KBS/VCI, VnExpress-Vietstock-CafeF, DNSE WebSocket)

↓ Thu thập và chuẩn hóa thông điệp ↓

MinIO Bronze (raw Parquet, truy vết theo nguồn/ngày ingest) → **Polars + kiểm tra schema**
→ **MinIO Silver** (cleaned/conformed Parquet)

↓ Great Expectations và quy tắc nghiệp vụ ↓

ClickHouse Gold serving ↔ **MinIO Gold Parquet** → **dbt / FastAPI** → **React Dashboard + Superset**

Streaming song song: DNSE/demo tick → Kafka topic dnse-trades-raw → ClickHouse Kafka Engine + Materialized View → bảng raw → Python consumer (15s) → fact_realtime_vwap.

Alert Engine song song: user_alerts (Postgres, 60s) → truy vấn Gold mới nhất → kiểm tra cooldown trên fact_alert_event → gửi cảnh báo + ghi log.

Airflow: điều phối DAG batch 16 task lúc 18:00, thứ Hai-Sáu; retry 1 lần, giới hạn một active run.

Fig. 1. Kiến trúc Data Lakehouse được triển khai trong đề tài, gồm nhánh batch và hai nhánh xử lý liên tục độc lập với lịch Airflow.

Materialized View đưa đủ 60 dòng vào bảng raw; consumer tổng hợp ra 15 dòng phút, với session_volume của VCB tăng tích lũy đúng quy luật (460 → 940 → 1.440 → 1.960 → 2.500) qua các phút liên tiếp; chạy lại consumer lần thứ hai trên cùng dữ liệu vẫn giữ đúng 15 dòng, xác nhận tính idempotent.

E. Alert Engine: pipeline cảnh báo event-driven

Alert Engine tách hai loại dữ liệu có đặc tính khác nhau: user_alerts trong PostgreSQL lưu cấu hình cảnh báo (ticker, loại điều kiện, ngưỡng, kênh gửi, cooldown_minutes, is_active) — bảng này có tính giao dịch cao, thường được thêm/sửa/tắt; còn fact_alert_event trong ClickHouse chỉ lưu lịch sử các lần điều kiện được kích hoạt, phục vụ truy vấn nhanh theo thời gian và làm cơ sở cho việc chống spam.

Sáu loại điều kiện được hỗ trợ: PRICE_ABOVE, PRICE_BELOW, RSI_ABOVE, RSI_BELOW, BB_BREAK và VWAP_DEVIATION. Một rule có thể đặt ticker = "ALL"; khi đó engine truy vấn toàn bộ mã đang thuộc dim_stock với exchange = 'HOSE' và mở rộng rule này thành một rule cụ thể cho từng mã trước khi đánh giá, thay vì phải tạo hàng trăm dòng cấu hình thủ công. Với mỗi rule đã mở rộng, engine lấy giá trị mới nhất từ fact_daily_price (giá, RSI14, Bollinger Band) và fact_realtime_vwap (độ lệch giá so với VWAP phiên), so với ngưỡng; nếu điều kiện thỏa, engine truy vấn fact_alert_event để kiểm tra tổ hợp user_id + ticker + condition_type đã được gửi trong cooldown_minutes gần nhất chưa. Nếu chưa, hệ thống gửi cảnh báo qua Telegram và ghi log với is_sent = 1; nếu đã có, hệ thống bỏ qua việc gửi nhưng vẫn ghi log với is_sent = 0

để giữ đầy đủ lịch sử kiểm toán, kể cả những lần bị chặn do cooldown.

Engine chạy trong một service Docker riêng (restart: unless-stopped), lặp mỗi 60 giây, độc lập với lịch batch của Airflow; DAG chính cũng có hai task init_user_alerts và check_alerts như một lớp kiểm tra dự phòng theo lịch ngày. Mỗi vòng quét toàn bộ 403 mã HOSE với 4 rule mặc định tương ứng 1.612 lượt kiểm tra. Toàn bộ sự kiện hiện có is_sent = 0 vì môi trường thử nghiệm chưa cấu hình TELEGRAM_BOT_TOKEN; đây là giới hạn được nêu lại ở Mục IV-B.

Khi theo dõi service chạy liên tục qua nhiều giờ, log cho thấy số lượt bị chặn bởi cooldown luôn bằng 0 dù cùng một điều kiện lặp lại mỗi vòng — một dấu hiệu vận hành bất thường. Nguyên nhân là câu truy vấn cooldown ban đầu chỉ tính các sự kiện có is_sent = 1; vì kênh Telegram chưa cấu hình nên is_sent luôn bằng 0, cooldown không bao giờ kích hoạt, và fact_alert_event tăng không kiểm soát theo từng vòng quét (từ khoảng 2.000 lên hơn 3.400 dòng chỉ trong một giờ). Khắc phục bằng cách tính cooldown trên mọi sự kiện đã ghi log, không phân biệt is_sent, đúng với ý định ban đầu là chặn lặp theo tổ hợp user_id + ticker + condition_type bất kể có gửi được thông báo hay không. Sau khi sửa và khởi động lại service, hai vòng quét liên tiếp cho cùng kết quả skipped_cooldown = 120, xác nhận cơ chế hoạt động đúng và tăng trưởng của bảng sự kiện đã được chặn lại.

F. Điều phối, chất lượng và tính toán

DAG stock_lakehouse_daily gồm 16 task, chạy lúc 18:00 từ thứ Hai đến thứ Sáu theo múi giờ Asia/Ho_Chi_Minh. Bốn nhánh ingest và Sil-

ver chạy song song theo phụ thuộc; tất cả phải qua quality gate trước khi migrate schema, nạp ClickHouse, xuất Gold Parquet, sinh snapshot cho frontend, rồi khởi tạo/kiểm tra cảnh báo ở hai task cuối. Airflow cung cấp retry và lịch sử task, giúp thay thế chuỗi lệnh chạy tay khó truy vết [2].

Quality gate kiểm tra cột bắt buộc, null, khóa trùng, giá dương, volume không âm và quan hệ $high \geq open, close, low, low \leq open, close$. Profile kiểm tra mã, tên, sàn, ngành và số cổ phiếu lưu hành; tin tức kiểm tra URL duy nhất, tiêu đề, nội dung, nguồn và thời gian xuất bản. Các kiểm tra được hiện thực bằng Polars và Great Expectations [5]. dbt bổ sung staging, model chỉ báo và test khóa duy nhất [6]; các model hiện materialize dạng view, là một đánh đổi tốc độ-đơn giản được nêu ở Mục IV-B.

Gold tính biến động giá, SMA20, EMA12/26, MACD, RSI14, Bollinger Bands, trung bình khối lượng 20 phiên và các cờ quá mua, quá bán, breakout/breakdown; các cờ và ngưỡng này được Alert Engine tái sử dụng làm điều kiện cảnh báo. Tin tức được liên kết với ticker theo thực thể và chấm điểm bằng từ điển tiếng Việt trong khoảng $[-1, 1]$. Với trade tick, VWAP phiên tại thời điểm t được tính:

$$VWAP_t = \frac{\sum_{i=1}^t P_i V_i}{\sum_{i=1}^t V_i}, \quad (1)$$

trong đó P_i và V_i lần lượt là giá và khối lượng giao dịch; công thức này được hiện thực bằng cum_sum của Polars theo từng cặp mã-ngày, dùng chung cho cả mô-đun demo và consumer Kafka.

G. Lớp phục vụ và dashboard

FastAPI cung cấp 10 endpoint: health check, danh sách mã HOSE, nến và chỉ báo theo mã, tổng quan thị trường, danh sách/chuỗi VWAP theo mã, tín hiệu kỹ thuật toàn thị trường, cảm xúc tin tức theo mã, lịch sử cảnh báo và trạng thái pipeline (tổng hợp từ row count các bảng Gold và báo cáo Great Expectations gần nhất). API realtime ưu tiên ClickHouse, nhưng tự đọc Bronze DNSE mới hơn khi serving chưa được cập nhật, giảm khả năng dashboard hiển thị snapshot cũ.

Frontend gồm bảy màn hình: Tổng quan thị trường, Chi tiết cổ phiếu, Bộ lọc tín hiệu kỹ thuật, Realtime VWAP, Phân tích tin tức, Lịch sử cảnh báo và Theo dõi pipeline; toàn bộ bảy màn hình hiện đã gọi API thật, với dữ liệu mẫu tĩnh trong mockData.ts chỉ còn vai trò fallback khi gọi API thất bại. Ngoài frontend tự xây, một dashboard Superset (“Stock Lakehouse Gold Overview”) được tạo qua REST API của Superset với ba chart truy vấn trực tiếp ClickHouse Gold: xu hướng VNIN-DEX theo ngày, top-10 mã theo giá trị giao dịch,

và điểm cảm xúc tin tức trung bình theo ngày; việc tạo dashboard qua API đòi hỏi liên kết rõ chart.dashboards và một position_json đúng cấu trúc lưới (ROOT → GRID → ROW → CHART), nếu thiếu sẽ dẫn tới lỗi runtime phía client dù bản ghi chart/dashboard vẫn được tạo thành công ở tầng API.

Superset và Grafana phục vụ hai mục đích khác nhau và không thay thế nhau: Superset trả lời câu hỏi nghiệp vụ trên Gold (giá, thanh khoản, sentiment), còn Grafana — đúng vai trò đã định nghĩa từ đầu trong tài liệu kiến trúc nhưng ban đầu chỉ là một service trống trong Docker Compose — trả lời câu hỏi vận hành: pipeline có đang chạy đúng không, dữ liệu có tươi không, Kafka có trễ không, cảnh báo nào đang nổ. Dashboard “Stock Lakehouse Ops Monitor” được provision hoàn toàn bằng cấu hình khai báo (datasource YAML và dashboard JSON đặt trong docker/grafana/provisioning/), không cần thiết lập tay: hai datasource được tạo tự động, gồm ClickHouse Gold (qua plugin grafana-clickhouse-datasource, tự cài bằng biến môi trường GF_INSTALL_PLUGINS) và Postgres chứa metadata Airflow. Bảy panel hiển thị số cảnh báo trong 24 giờ, độ trễ Kafka trung bình trên 60 tick gần nhất, độ tuổi của dữ liệu giá mới nhất, tổng số sự kiện cảnh báo, độ tươi và số dòng của từng bảng Gold, phân bố cảnh báo theo loại điều kiện, và lịch sử các DAG run của Airflow đọc trực tiếp từ bảng dag_run trong Postgres. Khi kiểm thử bằng trình duyệt headless, panel Postgres ban đầu báo lỗi kết nối vì cấu hình provisioning đặt nhầm host vào jsonData thay vì trường url ở cấp cao nhất theo đúng schema của datasource Postgres tích hợp sẵn của Grafana; sau khi sửa, cả bảy panel trả về dữ liệu thật và được xác nhận bằng ảnh chụp giao diện.

III. Kết quả thực hiện

A. Kết quả dữ liệu và chức năng

Số liệu trong Bảng II được truy vấn trực tiếp từ ClickHouse ngày 28/06/2026. Hệ thống đã tạo đủ bốn dimension và sáu bảng fact theo DDL, bao gồm fact_alert_event cho luồng cảnh báo. MinIO có đủ các bucket Bronze, Silver và Gold.

Dashboard Tổng quan cho phép xem VNIN-DEX/VN30, thanh khoản, độ rộng thị trường, top tăng/giảm, top thanh khoản và hiệu suất ngành. Màn hình Chi tiết cho phép tìm trong 403 mã HOSE, xem nến, tin tức theo mã và bật/tắt SMA, EMA, Bollinger Bands, RSI, MACD. Màn hình VWAP, Tín hiệu kỹ thuật, Tin tức và Lịch sử cảnh báo đều đọc dữ liệu thật từ Gold; riêng Lịch sử cảnh báo hiện hiển thị đúng thực trạng is_sent

TABLE II
Kết quả kiểm chứng trên Gold

Bảng/chỉ tiêu	Số lượng	Phạm vi
dim_stock	403	403 mã, chỉ HOSE
dim_sector	30	Nhóm ngành
dim_date	4.018	Chiều ngày
dim_index	2	VNINDEX, VN30
fact_daily_price	12.963	12/12/2024–26/06/2026
fact_market_index	66	VN30: đúng 30 mã/ngày
fact_news_sentiment	640	158 mã, 12-26/06/2026
fact_realtime_vwap	15	3 mã, qua Kafka, idempotent
fact_alert_event	5.089	403 mã, tăng trưởng bị chặn bởi cooldown

= 0 cho mọi sự kiện do chưa cấu hình kênh gửi thật.

B. Kiểm thử và khả năng vận hành

Project định nghĩa 164 test tự động cho ingestion, transform, quality, DDL, loader, dbt, Airflow, API, DNSE, Kafka consumer, VN30 constituents và Alert Engine. Lần chạy ngày 28/06/2026 đạt 164/164 test pass; công cụ lint tĩnh ruff không còn lỗi trên toàn repo. So với phiên bản trước (123/129 pass), các test còn lệch phạm vi HOSE/HNX/UPCOM và lỗi credential MinIO đã được rà soát và đồng bộ lại với hành vi thực tế của code, thay vì sửa test để che giấu một thay đổi hành vi không chủ đích.

Mã nguồn và cấu hình triển khai nằm trong repository của đề tài, gồm src/, scripts/, dags/, sql/, dbt/, frontend/, tests/ và docker-compose.yml với 12 service. Bộ cài được tái lập bằng Docker Compose và môi trường Python quản lý bởi uv; hai tiến trình nền (alert-engine, realtime-vwap-consumer) dùng cùng một Dockerfile tối giản chỉ gồm Python và uv, không phụ thuộc image Airflow.

IV. Đánh giá hiệu quả

A. Giá trị mang lại

So với cách tải tệp và xử lý thủ công, pipeline tạo một đường dữ liệu có lịch chạy, dependency, retry, quality gate và lịch sử thực thi. Bronze giữ dữ liệu gốc nên có thể điều tra sai lệch; Silver tạo schema thống nhất; Gold tách logic phân tích khỏi frontend. Parquet dạng cột giảm lượng dữ liệu đọc, còn partition pruning theo mã/tháng/ngày giới hạn số file cần quét [4]. ClickHouse phù hợp với truy vấn tổng hợp và chuỗi thời gian; API che giấu chi tiết kho dữ liệu khỏi giao diện.

Hai bổ sung gần nhất minh họa thêm giá trị của thiết kế data engineer: thứ nhất, việc tách Kafka Engine (chỉ làm passthrough) khỏi tầng

tính toán Python giúp tránh một lớp lỗi cụ thể — Materialized View tự tham chiếu alias và thu hẹp session_vwap thành vwap_lm nếu cố gắng tổng hợp trực tiếp trong SQL — đồng thời tái sử dụng được hàm tính VWAP đã kiểm thử ở nơi khác. Thứ hai, việc tách bảng cấu hình (Postgres) khỏi bảng sự kiện (ClickHouse) trong Alert Engine cho phép thay đổi rule mà không ảnh hưởng lịch sử, và cơ chế cooldown dựa trên truy vấn lịch sử thay vì trạng thái trong tiến trình giúp việc chống spam đúng cả khi engine được khởi động lại.

Thiết kế MinIO cho cả ba lớp tạo ra hai lợi ích: dữ liệu phân tích không bị khóa trong ClickHouse và có thể tái nạp serving khi cần; mặt khác, ClickHouse không phải gánh vai trò lưu trữ raw. Việc lọc HOSE ngay từ listing, transform và câu truy vấn API làm giảm nguy cơ trộn sần. Các khóa ticker--trading_date và index--trading_date cùng quality gate giúp ngăn trùng lặp và dữ liệu giả phi logic; với realtime VWAP, việc DROP PARTITION theo ngày trước khi ghi lại đảm bảo idempotency mà không cần khóa unique ở ClickHouse.

B. Hạn chế và rủi ro

Hiệu quả hiện mới được chứng minh bằng tính đúng cấu trúc, số bản ghi, test và khả năng chạy end-to-end với dữ liệu demo; project chưa có benchmark độ trễ, throughput hoặc chi phí ở quy mô dữ liệu thị trường thật trong giờ giao dịch. Loader Gold cho các bảng batch đang truncate rồi nạp lại toàn bộ, phù hợp quy mô hiện tại nhưng sẽ tốn thời gian khi dữ liệu tăng; nên chuyển sang incremental/upsert theo partition như cách realtime VWAP consumer đã áp dụng.

DNSE đã có thể publish lên Kafka thật, nhưng luồng end-to-end với tick thị trường thật (ngoài dữ liệu demo) chỉ chạy được trong giờ giao dịch và chưa được kiểm chứng tại thời điểm viết báo cáo. Kênh gửi cảnh báo qua email mới ở dạng log placeholder, chưa gửi SMTP thật; toàn bộ sự kiện cảnh báo hiện có is_sent = 0 vì thiếu cấu hình Telegram trong môi trường thử nghiệm, và schema hiện tại chỉ có một cờ is_sent nên không phân biệt được giữa “bị chặn do cooldown” và “gửi thất bại do thiếu cấu hình kênh”. Sentiment tin tức vẫn dựa trên từ điển nên chưa xử lý tốt phủ định, ngữ cảnh hoặc mỉa mai; có sẵn một pipeline huấn luyện PhoBERT cho hướng nâng cấp này nhưng chưa được tích hợp vào luồng Gold, nằm ngoài phạm vi tập trung data engineering của báo cáo. dbt model vẫn materialize dạng view; Superset dashboard mới có ba chart cơ bản và Grafana mới có bảy panel, cả hai chưa đủ phong phú theo roadmap đầy đủ của tuần 4. Panel độ trễ Kafka trên Grafana có thể trả giá trị âm khi dùng dữ

liệu demo vì `trade_ts` của tick giả lập được đặt theo giờ mở phiên mô phỏng, sớm hơn thời điểm `ingested_at` thật khi publish thử ngoài giờ; với tick DNSE thật, `trade_ts` luôn không muộn hơn `ingested_at` nên độ trễ sẽ không âm. Các bí mật API hiện được đọc từ biến môi trường, nhưng khi triển khai thật nên chuyển sang secret manager, xoay khóa và tách quyền truy cập.

V. Kết luận

Đề tài đã triển khai được nền tảng Data Lakehouse cho dữ liệu chứng khoán HOSE từ nguồn đến dashboard, mở rộng từ một pipeline batch thuần túy sang một hệ thống có cả streaming, alerting và giám sát vận hành thật: thu thập đa nguồn, lưu Parquet ba lớp trên MinIO, kiểm soát chất lượng, mô hình Gold trên ClickHouse, điều phối Airflow, mô hình dbt, FastAPI, React, Superset và Grafana. Đóng góp có tính mới trong phạm vi đề án là (1) một luồng Kafka Engine thật cho VWAP, thiết kế lại để tránh lỗi tổng hợp trực tiếp trong Materialized View và đảm bảo idempotent bằng chiến lược DROP PARTITION; (2) một Alert Engine event-driven tách cấu hình khỏi lịch sử sự kiện, hỗ trợ rule áp dụng cho toàn bộ vũ trụ mã bằng giá trị `ticker = "ALL"`, cùng một lỗi cooldown thật được phát hiện và sửa trong quá trình vận hành liên tục; (3) việc áp dụng partition theo hành vi truy vấn riêng cho từng dataset, tách storage layer khỏi serving layer; và (4) một dashboard Grafana giám sát vận hành được provision hoàn toàn bằng cấu hình khai báo, tách biệt khỏi dashboard phân tích nghiệp vụ trên Superset. Số liệu thực tế 403 mã HOSE, 12.963 dòng giá và hơn 5.000 sự kiện cảnh báo cho thấy hệ thống đã vượt mức mô phỏng cấu trúc đơn thuần.

Hướng phát triển gần nhất, theo đúng góc nhìn data engineer, là: chuyển loader Gold batch sang incremental/upsert; bổ sung benchmark độ trễ và throughput cho luồng Kafka khi chạy với tick thị trường thật; thêm cờ phân biệt rõ lý do không gửi được cảnh báo; chuyển dbt model sang table/incremental khi khối lượng dữ liệu tăng; và mở rộng dashboard Grafana đã có sang cảnh báo chủ động (Grafana Alerting) khi task Airflow treo quá SLA hoặc khi không còn tick Kafka mới trong một khoảng thời gian, thay vì chỉ hiển thị số liệu để người vận hành tự đọc. Khi pipeline HOSE ổn định, kiến trúc hiện tại cho phép mở rộng sàn hoặc nguồn mới bằng cấu hình và partition riêng mà không thay đổi lớp dashboard cốt lõi.

References

[1] Thịnh Vượng Tài Chính, "Vnstock Documentation," [Online]. Available: <https://vnstocks.com/docs/>. Accessed: Jun. 28, 2026.

[2] Apache Software Foundation, "Apache Airflow Documentation," [Online]. Available: <https://airflow.apache.org/docs/>. Accessed: Jun. 28, 2026.

[3] ClickHouse, Inc., "ClickHouse Documentation," [Online]. Available: <https://clickhouse.com/docs>. Accessed: Jun. 28, 2026.

[4] Apache Software Foundation, "Apache Parquet Documentation," [Online]. Available: <https://parquet.apache.org/docs/>. Accessed: Jun. 28, 2026.

[5] Great Expectations, "GX Core Documentation," [Online]. Available: <https://docs.greatexpectations.io/>. Accessed: Jun. 28, 2026.

[6] dbt Labs, "dbt Developer Hub," [Online]. Available: <https://docs.getdbt.com/>. Accessed: Jun. 28, 2026.